Updated Controller File Responsibilities (Backend)

Discusses the use case and distinction between controller files.

1️⃣ authController.js

Purpose:
Handles authentication and account entry logic.

When to put functions here:

- User registration
- User login
- Issuer login
- Password changes related to authentication/account access

Example functions

- `register`
- `login`
- `loginIssuer`
- `changePassword`

Rule of thumb:
If the feature is about **logging in, registering, or proving identity/access**, it belongs here.

2️⃣ accountController.js

Purpose:
Handles general user account and profile management.

When to put functions here:

- Fetching user info
- Fetching all users
- Updating profile details
- Uploading profile pictures

Example functions

- `getUserInfo`
- `getAllUsers`
- `updateProfile`
- `uploadProfilePicture`

Rule of thumb:
If the feature is about the **user's account details or profile**, but not login/auth itself, it belongs here.

3️⃣ issuerOnboardingController.js

Purpose:
Handles issuer onboarding, issuer application requirements, and issuer-submitted documents.

When to put functions here:

- Issuer registration
- Fetching issuer requirements by category
- Viewing submitted issuer documents

- Reviewing onboarding-related issuer files

Example functions

- `registerIssuer`
- `getIssuerRequirementsByCategory`
- `getIssuerDocuments`
- `viewIssuerDocument`

Rule of thumb:
If the feature is about **becoming an issuer**, submitting onboarding requirements, or viewing issuer application documents, it belongs here.

4️⃣ issuerCredentialController.js

Purpose:
Handles issuer-side credential lifecycle operations tied to creation, issuance, revocation, history, and issuer credential reads.

When to put functions here:

- Creating credential templates
- Issuing credentials
- Revoking credentials
- Fetching issuer templates
- Fetching issued credentials
- Checking if issuer has templates
- Fetching credential history

- Getting eligible users for a template

Example functions

- `Create_CredentialTemplate`
- `Issue_Credential`
- `revoke_Credential`
- `get_CredentialTemplatesByIssuerID`
- `get_IssuerIssuedCredentials`
- `check_IssuerHasTemplates`
- `Get_CredentialHistory`
- `getEligibleUsersForTemplate`
- `getCredentialTemplates`
- `getIssuerCredentials`

Rule of thumb:
If the feature is about the **issuer creating, issuing, revoking, or viewing credentials**, it belongs here.

5️⃣ adminIssuerController.js

Purpose:
Handles admin-side issuer lifecycle management.

When to put functions here:

- Viewing issuer lists by status

- Approving issuers
- Rejecting issuers
- Enabling/disabling issuers
- Fetching pending root/subordinate issuers
- Fetching admin-actionable issuer queues

Example functions

- `getPendingIssuers`
- `getAdminActionablePendingIssuers`
- `getApprovedIssuers`
- `getRejectedIssuers`
- `getDisabledIssuers`
- `getPendingRootIssuers`
- `getPendingSubordinateIssuers`
- `getPendingSubordinateIssuersAwaitingRootApproval`
- `getPendingSubordinateIssuersRootApproved`
- `Approve_Issuer`
- `Reject_Issuer`
- `Disable_Issuer`
- `Enable_Issuer`

Rule of thumb:
If the feature is about the **admin managing issuers and issuer statuses**, it belongs here.

6️⃣ rootAuthorityController.js

Purpose:
Handles root authority governance over subordinate issuers.

When to put functions here:

- Viewing subordinate issuers under a root authority
- Approving subordinate issuers at root level
- Rejecting subordinate issuers at root level
- Filtering subordinate issuers by status

Example functions

- `getPendingSubordinatesForRoot`
- `getAllSubordinatesForRoot`
- `getRootApprovedSubordinatesForRoot`
- `getRootRejectedSubordinatesForRoot`
- `getApprovedSubordinatesForRoot`
- `getDisabledSubordinatesForRoot`
- `rootApproveSubordinateIssuer`
- `rootRejectSubordinateIssuer`

Rule of thumb:
If the feature is about a **root authority managing subordinate issuers**, it belongs here.

7️⃣ adminAuditController.js

Purpose:
Handles admin-only logs, audit trails, security-sensitive admin actions, and user status management.

When to put functions here:

- Admin password change with blockchain audit log
- Viewing status logs
- Viewing issuer/user status logs
- Viewing issuer decision logs
- Enabling/disabling users
- Security/audit-related admin actions

Example functions

- `changeAdminPasswordSecure`
- `getAllStatusLogs`
- `getAllIssuerStatusLogs`
- `getAllUserStatusLogs`
- `getIssuerStatusLogs`
- `getUserStatusLogs`
- `getAllIssuerDecisionLogs`
- `getAllIssuerApprovalLogs`

- `getAllIssuerRejectionLogs`
- `getIssuerDecisionLogs`
- `Disable_User`
- `Enable_User`

Rule of thumb:
If the feature is about **logs, audit trail, admin security actions, or admin-controlled user status changes**, it belongs here.

8️⃣ adminDashboardController.js

Purpose:
Handles admin dashboard aggregation and dashboard-wide maintenance actions.

When to put functions here:

- Fetching admin-wide credential dashboard data
- Triggering expiry checks
- Dashboard summary/overview operations
- Admin dashboard data aggregation

Example functions

- `getAllIssuerCredentialsAdminController`
- `triggerExpiryCheck`

Rule of thumb:
If the feature is specifically for the **Admin Dashboard overview or maintenance tools**, it belongs here.

9️⃣ walletController.js

Purpose:
Handles student wallet behavior and wallet-specific credential display/state.

When to put functions here:

- Fetching wallet credentials
- Updating credential visibility
- Updating credential order
- Wallet-only credential presentation logic

Example functions

- `getUserCredentials`
- `updateCredentialVisibility`
- `updateCredentialOrder`

Rule of thumb:
If the feature is about **how credentials appear or behave in the student wallet**, it belongs here.

🔟 verifierController.js

Purpose:
Handles credential verification and verification logging/statistics.

When to put functions here:

- Verifying credentials
- Logging verification events
- Fetching verification logs

- Fetching verification stats

Example functions

- `Verify_Credential`
- `logVerificationEvent`
- `getAllVerificationLogs`
- `getVerificationLogById`
- `getVerificationStats`

Rule of thumb:
If the feature is about **checking whether a credential is valid**, or about **verification logs and stats**, it belongs here.

Quick Mental Model

Think of your controllers like backend workers with specialties:

Controller File	Responsibility
`authController`	Authentication / Access Entry
`accountController`	User Profile / Account Data
`issuerOnboardingController`	Issuer Application / Requirements / Documents
`issuerCredentialController`	Issuer Credential Operations
`adminIssuerController`	Admin Management of Issuers
`rootAuthorityController`	Root Authority over Subordinates

`adminAuditController`	Admin Logs / Audit / Security / User Status
`adminDashboardController`	Admin Dashboard Aggregation / Maintenance
`walletController`	Student Wallet Behavior
`verifierController`	Credential Verification

Example: Which controller should a new feature use?

Feature:
User wants to update their profile picture
➡ `accountController.js`

Feature:
Issuer submits onboarding documents during registration
➡ `issuerOnboardingController.js`

Feature:
Issuer wants to issue a certificate to a student
➡ `issuerCredentialController.js`

Feature:
Admin wants to reject an aspiring issuer
➡ `adminIssuerController.js`

Feature:
Root authority wants to approve a subordinate issuer
➡ `rootAuthorityController.js`

Feature:
Admin wants to view all issuer rejection logs
➡ `adminAuditController.js`

Feature:
Admin wants dashboard-wide credential counts
➡ `adminDashboardController.js`

Feature:
Student wants to hide a credential in wallet
➡ `walletController.js`

Feature:
Verifier scans a QR and checks validity
➡ `verifierController.js`

Feature:
User logs into the platform
➡ `authController.js`

Easy Rule for Future Development

When adding a new backend feature, ask:

1. Who is performing the action?

- user/account holder → maybe `accountController`
- issuer → maybe `issuerOnboardingController` or `issuerCredentialController`
- admin → maybe `adminIssuerController`, `adminAuditController`, or `adminDashboardController`
- root authority → `rootAuthorityController`
- verifier/public checker → `verifierController`

2. What is the action mainly about?

- login/auth → `authController`
- profile/account info → `accountController`
- onboarding/documents → `issuerOnboardingController`

- credential lifecycle → `issuerCredentialController`
- issuer governance → `adminIssuerController`
- subordinate governance → `rootAuthorityController`
- logs/audit/security → `adminAuditController`
- dashboard aggregation → `adminDashboardController`
- wallet display/state → `walletController`
- verification → `verifierController`

Suggested Rule of Thumb Per Layer

This helps avoid confusion next time:

- **Route file** = based on API/domain grouping
- **Controller file** = based on business responsibility
- **Service file** = shared data access / DB / helper logic
- **Chaincode** = blockchain source of truth logic

So:

- If you're deciding **which endpoint file** gets the route → check the **route responsibilities**
- If you're deciding **where the backend logic should live** → check the **controller responsibilities**